

Day 23 : Testing basics

1. Introduction to Flutter Testing Guide

Flutter provides a comprehensive testing framework that helps developers ensure the quality, reliability, and correctness of their applications. Testing helps identify bugs early, improve code quality, and maintain application stability.

2. Types of Testing in Flutter

A. Unit Testing

Unit tests verify the correctness of individual functions, methods, classes, or business logic without involving the user interface.

Examples:

- Calculator operations
- Form validation logic
- Model classes
- State management methods

B . Widget Testing

Widget tests verify whether a single widget renders and behaves correctly.

Examples:

- Button click functionality
- TextField input validation
- Widget visibility
- Navigation between screens

C. Integration Testing

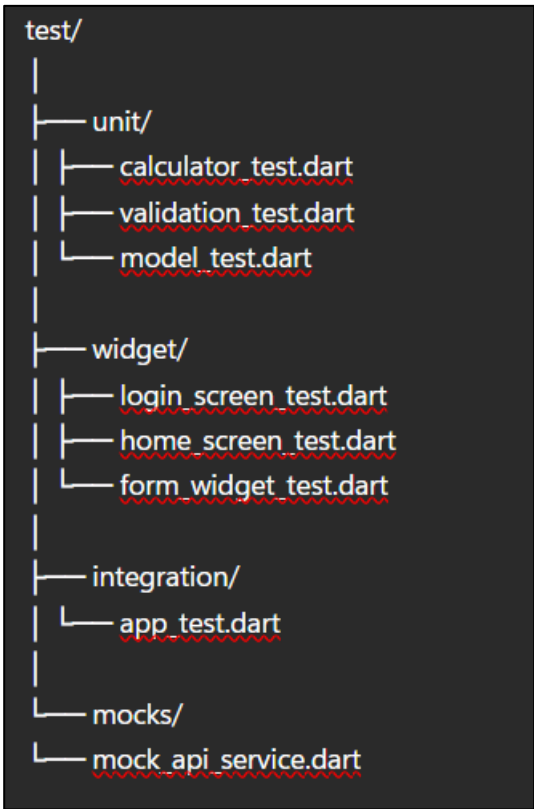
Integration tests verify the complete application flow by testing multiple widgets and services together.

Examples:

- User login flow
- API integration
- Complete app navigation

3. Project Structure for Testing

The Flutter project uses the following directory structure for test implementation:



4. Test Cases Implemented

Unit Test Cases

Test Case	Expected Result
Addition Function	Correct sum is returned
Email Validation	Valid email accepted
Password Validation	Invalid password rejected
Counter Increment	Counter value increases

Widget Test Cases

Test Case	Expected Result
Login Button Rendering	Button appears on screen
Text Field Input	Entered text is displayed
Button Click Event	Proper action is executed
Navigation Test	User navigates to next screen

Integration Test Cases

Test Case	Expected Result
User Login Flow	User logs in successfully
API Fetch Operation	Data is loaded correctly
Form Submission	Form is submitted successfully

5. Test Execution

The following command is used to run all tests:

```
flutter test
```

To run a specific test file:

```
flutter test test/calculator_test.dart
```

Tests can also be executed directly from Android Studio or Visual Studio Code.

6. Test Coverage

Test coverage measures the percentage of source code executed during testing.

Coverage report generation command:

```
flutter test --coverage
```

The generated report is stored in:

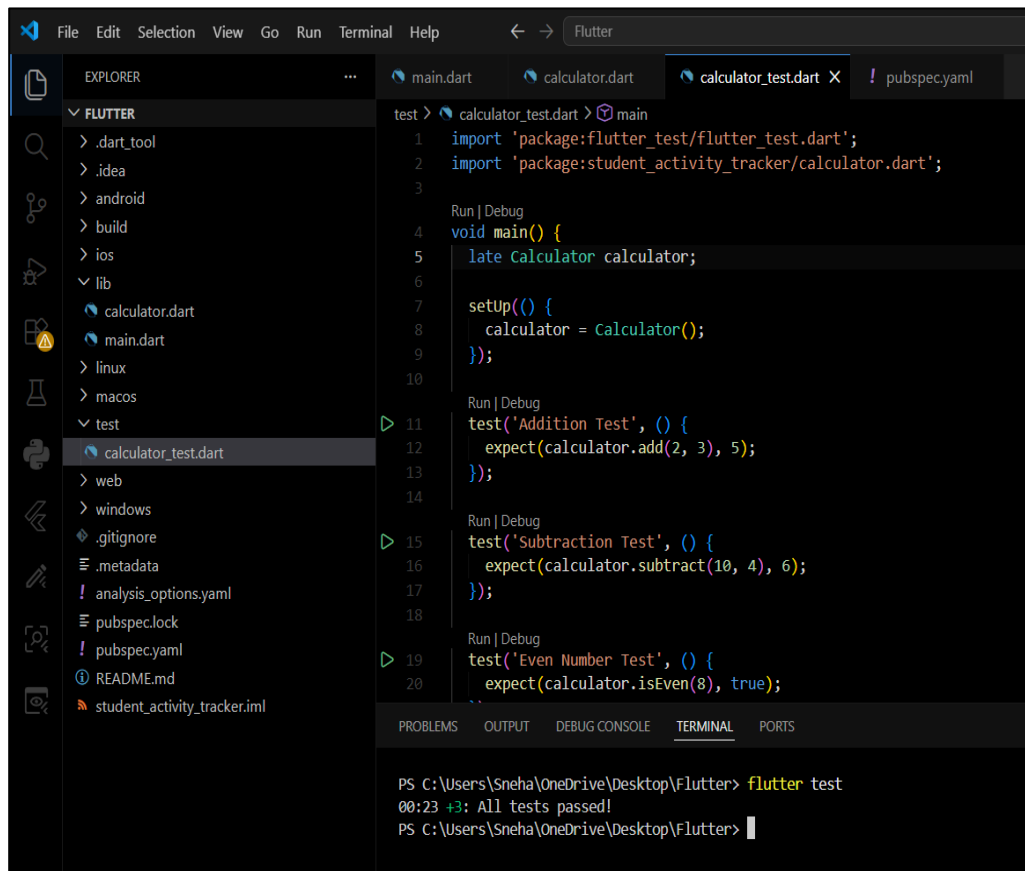
```
coverage/lcov.info
```

Higher test coverage ensures better application quality and reliability.

7. Tools and Technologies Used

- Flutter SDK
- Dart Programming Language
- Flutter Test Package
- Android Studio / Visual Studio Code
- Flutter DevTools
- Mocking Libraries (Mockito or Mocktail)

Unit testing :

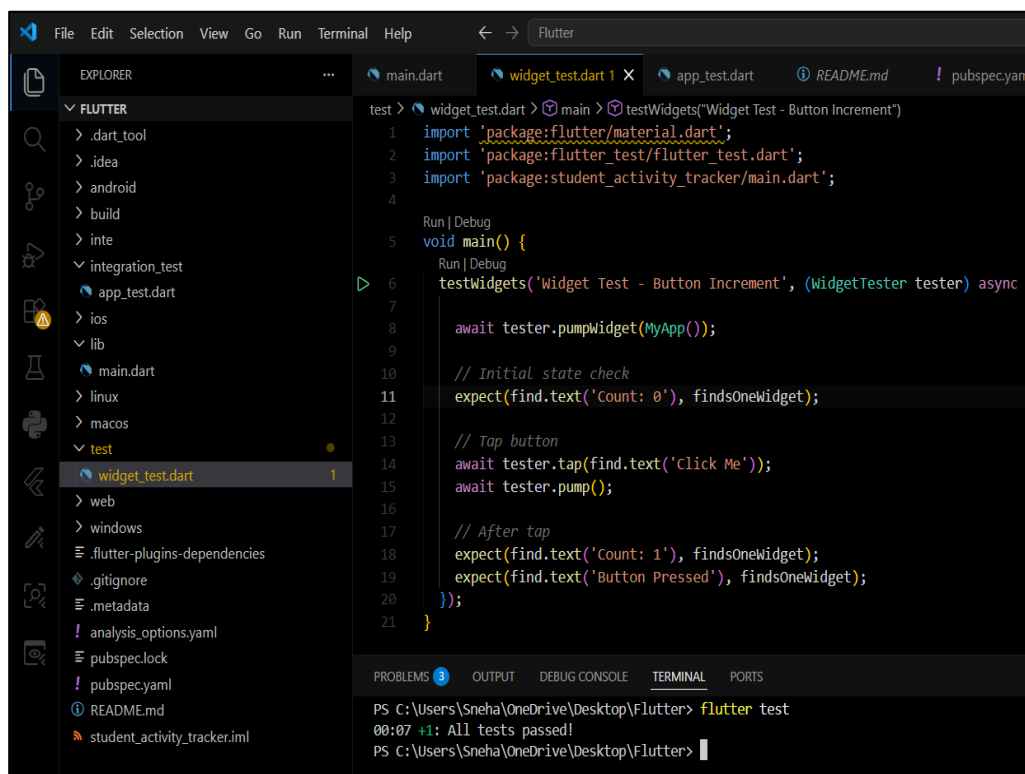


```
test > calculator_test.dart > main
1 import 'package:flutter_test/flutter_test.dart';
2 import 'package:student_activity_tracker/calculator.dart';
3
4 void main() {
5   late Calculator calculator;
6
7   setUp(() {
8     calculator = Calculator();
9   });
10
11   test('Addition Test', () {
12     expect(calculator.add(2, 3), 5);
13   });
14
15   test('Subtraction Test', () {
16     expect(calculator.subtract(10, 4), 6);
17   });
18
19   test('Even Number Test', () {
20     expect(calculator.isEven(8), true);
21   });
22 }
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
PS C:\Users\Sneha\OneDrive\Desktop\Flutter> flutter test
00:23 +3: All tests passed!
PS C:\Users\Sneha\OneDrive\Desktop\Flutter>
```

Widget Testing :



```
test > widget_test.dart > main > testWidgets("Widget Test - Button Increment")
1 import 'package:flutter/material.dart';
2 import 'package:flutter_test/flutter_test.dart';
3 import 'package:student_activity_tracker/main.dart';
4
5 void main() {
6   testWidgets('Widget Test - Button Increment', (WidgetTester tester) async {
7     await tester.pumpWidget(MyApp());
8
9     // Initial state check
10    expect(find.text('Count: 0'), findsOneWidget);
11
12    // Tap button
13    await tester.tap(find.text('Click Me'));
14    await tester.pump();
15
16    // After tap
17    expect(find.text('Count: 1'), findsOneWidget);
18    expect(find.text('Button Pressed'), findsOneWidget);
19  });
20 }
21 }
```

PROBLEMS 3 OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
PS C:\Users\Sneha\OneDrive\Desktop\Flutter> flutter test
00:07 +1: All tests passed!
PS C:\Users\Sneha\OneDrive\Desktop\Flutter>
```

Integration Testing :

```
test > widget_test.dart > main > testWidgets("Widget Test - Button Increment")
1  import 'package:flutter/material.dart';
2  import 'package:flutter_test/flutter_test.dart';
3  import 'package:student_activity_tracker/main.dart';
4
Run|Debug
5  void main() {
Run|Debug
6    testWidgets('Widget Test - Button Increment', (WidgetTester tester) async {
7
8      await tester.pumpWidget(MyApp());
9
10     // Initial state check
11     expect(find.text('Count: 0'), findsOneWidget);
12
13     // Tap button
14     await tester.tap(find.text('click Me'));
15     await tester.pump();
16
17     // After tap
18     expect(find.text('Count: 1'), findsOneWidget);
19     expect(find.text('Button Pressed'), findsOneWidget);
20
21   });
22 }
```

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Sneha\OneDrive\Desktop\Flutter> flutter test
00:07 +1: All tests passed!
PS C:\Users\Sneha\OneDrive\Desktop\Flutter> flutter test integration_test
00:01 +0: loading C:/Users/Sneha/OneDrive/Desktop/Flutter/integration_test/app_test.dart
02:53 +0: loading C:/Users/Sneha/OneDrive/Desktop/Flutter/integration_test/app_test.dart
✓ Built build/app/outputs/flutter-apk/app-debug.apk
02:56 +0: loading C:/Users/Sneha/OneDrive/Desktop/Flutter/integration_test/app_test.dart
03:02 +0: loading C:/Users/Sneha/OneDrive/Desktop/Flutter/integration_test/app_test.dart
03:10 +1: All tests passed!
PS C:\Users\Sneha\OneDrive\Desktop\Flutter>
```